Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho

PCC104 - Projeto e Análise de Algoritmos

Henrique Ulhôa de Faria
Professor: Pedro Silva

Ouro Preto
19 de outubro de 2026

Sumário

1 Introdução **2**
- 1.1 Especificações do problema . . . 2
- 1.2 Considerações iniciais . . . 2
- 1.3 Ferramentas utilizadas . . . 2
- 1.4 Especificações da máquina . . . 2
- 1.5 Instruções de compilação e execução . . . 3

2 Metodologia **4**
- 2.1 Análise dos enunciados . . . 4
- 2.2 Projeto e implementação dos algoritmos . . . 4
- 2.3 Análise de complexidade . . . 4
- 2.4 Testes e validação . . . 4
- 2.5 Organização e documentação . . . 4

3 Tarefa 1 – Multiplicação de Matrizes **5**
- 3.1 Descrição do problema . . . 5
- 3.2 Estratégia de solução . . . 5
  - 3.2.1 Tratamento de matrizes de ordem arbitrária . . . 5
- 3.3 Implementação . . . 6
  - 3.3.1 Estrutura de dados . . . 6
  - 3.3.2 Operações básicas . . . 6
  - 3.3.3 Funções auxiliares . . . 6
  - 3.3.4 Multiplicação recursiva . . . 7
  - 3.3.5 Função principal de multiplicação . . . 7
  - 3.3.6 Programa principal . . . 7
- 3.4 Análise de complexidade . . . 7
  - 3.4.1 Complexidade de tempo . . . 7
  - 3.4.2 Complexidade de espaço . . . 8
  - 3.4.3 Análise do preenchimento com zeros . . . 9
- 3.5 Experimentos e resultados . . . 9
  - 3.5.1 Metodologia dos testes . . . 9
  - 3.5.2 Casos de teste . . . 9
  - 3.5.3 Teste 01 – Matrizes de ordem 2 . . . 10
  - 3.5.4 Teste 02 – Caso base . . . 10
  - 3.5.5 Teste 03 – Ordem não potência de dois . . . 10
  - 3.5.6 Teste 04 – Matriz identidade . . . 10
  - 3.5.7 Análise dos resultados . . . 11
- 3.6 Considerações da tarefa . . . 11

4 Tarefa 2 – Título a definir **12**

5 Tarefa 3 – Título a definir **13**

6 Tarefa 4 – Título a definir **14**

7 Tarefa 5 – Título a definir **15**

8 Tarefa 6 – Título a definir **16**

9 Tarefa 7 – Título a definir **17**

10 Tarefa 8 – Título a definir **18**

11 Tarefa 9 – Título a definir **19**

12 Considerações Finais **20**

Lista de Tabelas

1 Casos de teste da multiplicação de matrizes. 9

Lista de Códigos Fonte

1 Definição do TAD Matriz. 6
2 Caso base da multiplicação recursiva. 7

Sumário

1 Introdução

Este relatório apresenta o desenvolvimento das atividades propostas no Trabalho Prático da disciplina PCC104 – Projeto e Análise de Algoritmos. O trabalho contempla a implementação, a análise de complexidade e a validação de algoritmos, com o objetivo de relacionar os conceitos teóricos estudados na disciplina às suas aplicações práticas.

As soluções são desenvolvidas na linguagem de programação C, considerando aspectos como organização do código, correção dos resultados, eficiência computacional e utilização de estruturas de dados adequadas.

1.1 Especificações do problema

O Trabalho Prático é composto por diferentes tarefas de implementação de algoritmos. Cada tarefa apresenta um problema específico, com requisitos de entrada, saída e processamento que devem ser respeitados.

Para cada problema, busca-se construir uma solução correta, analisar sua complexidade de tempo e espaço e verificar seu funcionamento por meio de testes. As especificações particulares de cada atividade são apresentadas nos respectivos capítulos deste relatório.

1.2 Considerações iniciais

O relatório está organizado em uma parte introdutória, uma seção de metodologia geral e capítulos dedicados individualmente às tarefas. Cada capítulo de tarefa apresenta a descrição do problema, a estratégia de solução, os aspectos relevantes da implementação, a análise de complexidade, os testes e os resultados obtidos.

O código-fonte é desenvolvido em C e organizado em arquivos separados, conforme as necessidades de cada atividade. A documentação é elaborada em LaTeX, utilizando a plataforma Overleaf.

1.3 Ferramentas utilizadas

As ferramentas empregadas no desenvolvimento e na validação das implementações incluem:

- **Linguagem C**: implementação dos algoritmos e das estruturas de dados.
- **Visual Studio Code**: edição e organização dos arquivos de código-fonte.
- **GCC**: compilação dos programas em C.
- **GNU Make**: automatização da compilação e da execução dos testes.
- **MSYS2 UCRT64**: ambiente utilizado para compilação e testes no Windows.
- **Git e GitHub**: controle de versões e armazenamento do projeto.
- **Overleaf e LaTeX**: elaboração e formatação do relatório.

1.4 Especificações da máquina

O desenvolvimento e os testes foram realizados em um computador com as seguintes características, a serem confirmadas:

- **Processador**: Intel I5 11° Geração.
- **Memória RAM**: 24 Ram.
- **Sistema operacional**: Windows 10.
- **Sistema operacional**: Ubuntu.
- **Compilador**: GCC no ambiente MSYS2 UCRT64 [informar versão utilizada].

1.5 Instruções de compilação e execução

Os programas são organizados em diretórios específicos para cada tarefa. Quando houver um arquivo `Makefile`, a compilação poderá ser realizada, a partir do diretório correspondente, com o comando:

Compilação com Make

```
make
```

Na Tarefa 1, também é possível compilar diretamente com o GCC:

Compilação da Tarefa 1

```
gcc -Wall -Wextra -Wpedantic -std=c11 -Iinclude src/main.c src/matriz.c -o tarefa01.exe
```

As principais opções utilizadas são:

- `-Wall`: habilita avisos comuns do compilador.
- `-Wextra`: habilita avisos adicionais.
- `-Wpedantic`: sinaliza construções que não seguem estritamente o padrão selecionado.
- `-std=c11`: seleciona o padrão C11.
- `-Iinclude`: informa o diretório dos arquivos de cabeçalho.
- `-o`: define o nome do arquivo executável.

Para executar a Tarefa 1 utilizando um arquivo de entrada, emprega-se:

Execução da Tarefa 1

```
./tarefa01.exe < tests/entrada01.txt
```

Para executar os testes automatizados definidos no `Makefile`, utiliza-se:

Testes automatizados

```
make test
```

Os comandos de compilação e execução das demais tarefas serão apresentados ou ajustados conforme suas implementações.

2 Metodologia

O desenvolvimento das atividades segue uma sequência de etapas que busca assegurar a correção dos algoritmos e permitir a análise de seu desempenho. Os procedimentos gerais são descritos a seguir; as particularidades de cada solução são detalhadas nos capítulos correspondentes.

2.1 Análise dos enunciados

Inicialmente, são identificados o objetivo de cada problema, os dados de entrada, os resultados esperados e as restrições estabelecidas. Essa etapa orienta a escolha da estratégia algorítmica e a definição das estruturas de dados necessárias.

2.2 Projeto e implementação dos algoritmos

Após a interpretação do problema, é definida uma estratégia de solução, considerando as técnicas estudadas na disciplina. A implementação é realizada em linguagem C, com separação de responsabilidades entre funções e arquivos quando apropriado.

A documentação de cada tarefa descreve as principais funções, as decisões de implementação e os casos especiais tratados. Trechos relevantes de código podem ser incluídos no relatório utilizando o ambiente `lstlisting` disponibilizado pelo modelo.

2.3 Análise de complexidade

A eficiência das soluções é estudada por meio da análise assintótica de tempo e espaço. São identificadas as operações predominantes e, para algoritmos recursivos, formuladas relações de recorrência. Quando aplicável, essas relações são resolvidas por técnicas como o Teorema Mestre ou árvores de recorrência.

A análise teórica considera as hipóteses necessárias a cada algoritmo, que são explicitadas nas respectivas tarefas.

2.4 Testes e validação

As implementações são verificadas por meio de casos de teste que contemplam entradas simples, casos representativos e situações particulares previstas nos enunciados. Os resultados produzidos são comparados com as saídas esperadas.

Quando disponível, um `Makefile` automatiza a compilação e a execução dos testes. A documentação apresenta os casos utilizados e registra os resultados observados, distinguindo a verificação de correção de eventuais medições experimentais de desempenho.

2.5 Organização e documentação

O projeto utiliza Git para o versionamento dos arquivos de implementação e teste. Este relatório reúne a descrição das soluções, as análises de complexidade e os resultados obtidos, mantendo uma estrutura uniforme para facilitar a comparação e a consulta às diferentes tarefas.

3 Tarefa 1 – Multiplicação de Matrizes

3.1 Descrição do problema

A primeira tarefa consiste na implementação de um algoritmo para realizar a multiplicação de duas matrizes quadradas de ordem n, utilizando a técnica de divisão e conquista. Dadas duas matrizes A e B, ambas de dimensão $n \times n$, o objetivo é calcular a matriz resultante $C = A \cdot B$, também de ordem n. Cada elemento da matriz resultante é definido por:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}, \qquad 1 \leq i, j \leq n. \tag{1}$$

A entrada do programa é composta por um inteiro positivo n, que representa a ordem das matrizes, seguido dos n^2 elementos da matriz A e dos n^2 elementos da matriz B. A saída consiste nos elementos da matriz resultante C, organizados em n linhas e n colunas. A solução deve utilizar uma abordagem recursiva baseada em divisão e conquista, decompondo o problema original em multiplicações de submatrizes de menor dimensão.

3.2 Estratégia de solução

Para resolver o problema, foi utilizada a técnica de divisão e conquista, que consiste em dividir um problema em subproblemas menores, resolvê-los recursivamente e combinar seus resultados para obter a solução original.

Na multiplicação de matrizes, essa estratégia é aplicada dividindo cada matriz quadrada em quatro submatrizes de mesma dimensão. Considerando duas matrizes A e B, de ordem n, suas decomposições podem ser representadas por:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}. \tag{2}$$

Cada submatriz possui dimensão $\frac{n}{2} \times \frac{n}{2}$, considerando inicialmente que n é uma potência de dois.

A matriz resultante $C = A \cdot B$ também pode ser representada por quatro submatrizes:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}. \tag{3}$$

Os blocos da matriz resultante são calculados pelas seguintes expressões:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \tag{4}$$
$$C_{12} = A_{11}B_{12} + A_{12}B_{22}, \tag{5}$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}, \tag{6}$$
$$C_{22} = A_{21}B_{12} + A_{22}B_{22}. \tag{7}$$

Dessa forma, cada etapa da recursão realiza oito multiplicações de submatrizes de ordem $n/2$, além das operações de soma e combinação dos resultados parciais.

O caso base da recursão ocorre quando as matrizes possuem ordem $n = 1$. Nesse caso, a multiplicação é realizada diretamente entre seus únicos elementos, encerrando a chamada recursiva.

Após o cálculo dos quatro blocos resultantes, eles são combinados para formar a matriz produto da etapa atual.

3.2.1 Tratamento de matrizes de ordem arbitrária

Como a divisão recursiva utiliza submatrizes de mesma dimensão, foi implementado um procedimento para tratar matrizes cuja ordem não é uma potência de dois.

Inicialmente, determina-se a menor potência de dois maior ou igual à ordem original n:

$$m = 2^{\lceil \log_2 n \rceil}. \tag{8}$$

Em seguida, são criadas duas matrizes auxiliares de ordem m, preenchidas inicialmente com zeros. Os elementos das matrizes originais são copiados para as posições correspondentes dessas matrizes auxiliares. Esse procedimento, denominado preenchimento com zeros (*zero-padding*), permite que as matrizes sejam divididas sucessivamente em quatro blocos de mesma dimensão. Após a multiplicação recursiva, são extraídos os $n \times n$ elementos da região superior esquerda da matriz resultante, obtendo-se o produto correspondente às matrizes originais. Assim, a solução permite multiplicar matrizes quadradas de qualquer ordem positiva, mantendo a estratégia recursiva de divisão e conquista.

3.3 Implementação

A implementação foi desenvolvida na linguagem C, utilizando um Tipo Abstrato de Dados (TAD) para representar e manipular matrizes quadradas.

O código foi organizado em três arquivos principais: `matriz.h`, responsável pela definição da estrutura e declaração das funções; `matriz.c`, que contém a implementação das operações sobre matrizes; e `main.c`, responsável pela leitura dos dados, execução da multiplicação e apresentação do resultado.

3.3.1 Estrutura de dados

A estrutura `Matriz` armazena a ordem da matriz e um ponteiro para seus elementos, representados por um vetor de ponteiros para linhas alocadas dinamicamente.

```
typedef struct {
    int n;
    int **dados;
} Matriz;
```

Código 1: Definição do TAD Matriz.

O campo `n` representa a quantidade de linhas e colunas, enquanto `dados` permite acessar os elementos por meio da expressão `m->dados[i][j]`.

A alocação dinâmica permite criar matrizes de diferentes dimensões durante a execução, o que é necessário para o funcionamento da abordagem recursiva.

3.3.2 Operações básicas

Foram implementadas funções para criação, liberação de memória, leitura e impressão de matrizes.

A função `createMatriz` recebe a ordem da matriz e realiza a alocação dinâmica de sua estrutura e de seus elementos, inicializados com zero.

A função `deleteMatriz` libera a memória alocada, enquanto `readMatriz` e `printMatriz` realizam, respectivamente, a leitura dos elementos e a apresentação da matriz.

Essas operações são utilizadas tanto pelo programa principal quanto pelas funções auxiliares da multiplicação.

3.3.3 Funções auxiliares

Para implementar a estratégia de divisão e conquista, foram desenvolvidas três funções auxiliares:

- `subMatriz`: cria uma nova matriz contendo uma região quadrada da matriz original, definida pela posição inicial e pela dimensão desejada.
- `somaMatriz`: recebe duas matrizes de mesma ordem e retorna uma nova matriz contendo a soma de seus elementos correspondentes.
- `juntaMatriz`: combina quatro submatrizes em uma única matriz quadrada, posicionando cada bloco no quadrante correspondente.

Essas funções permitem dividir as matrizes de entrada, combinar os produtos parciais e reconstruir o resultado de cada chamada recursiva.

3.3.4 Multiplicação recursiva

A função `multiplicaRecursivo` implementa a etapa principal do algoritmo de divisão e conquista.

A função recebe duas matrizes de mesma ordem e retorna uma nova matriz correspondente ao produto entre elas.

O caso base ocorre quando a ordem das matrizes é igual a um. Nessa situação, a multiplicação é realizada diretamente, conforme apresentado no Código 2.

```
if (n == 1) {
    Matriz *C = createMatriz(1);

    if (C != NULL)
        C->dados[0][0] = A->dados[0][0] * B->dados[0][0];

    return C;
}
```

Código 2: Caso base da multiplicação recursiva.

Para matrizes de ordem superior a um, são criadas quatro submatrizes de cada operando. Em seguida, são realizadas oito chamadas recursivas para calcular os produtos necessários à construção dos quatro blocos da matriz resultante. Os produtos são somados dois a dois, conforme as expressões apresentadas na Seção 3, e os quatro blocos resultantes são combinados pela função `juntaMatriz`. A implementação também realiza a liberação das matrizes temporárias após sua utilização, evitando que as alocações intermediárias permaneçam ocupando memória desnecessariamente.

3.3.5 Função principal de multiplicação

A função `multiplicaMatriz` funciona como interface para a multiplicação recursiva.

Inicialmente, verifica as matrizes recebidas e determina a menor potência de dois maior ou igual à ordem original.

Quando necessário, são criadas matrizes auxiliares maiores, preenchidas com zeros, para permitir a divisão recursiva em quadrantes de mesma dimensão.

Após a execução de `multiplicaRecursivo`, a função constrói a matriz resultante com a dimensão original, copiando os elementos correspondentes e liberando as estruturas temporárias.

Dessa forma, o programa permite multiplicar matrizes quadradas de ordem arbitrária, mantendo a lógica recursiva concentrada em uma função específica.

3.3.6 Programa principal

O arquivo `main.c` coordena a execução do programa, realizando a leitura da ordem das matrizes, a criação das estruturas, a leitura dos elementos e a chamada da função `multiplicaMatriz`.

Após a multiplicação, o resultado é impresso e a memória alocada é liberada.

A implementação verifica possíveis falhas de leitura e alocação, encerrando a execução quando não é possível prosseguir corretamente.

3.4 Análise de complexidade

A análise de complexidade do algoritmo considera o número de operações realizadas durante a multiplicação recursiva de duas matrizes quadradas de ordem n, bem como a quantidade de memória auxiliar necessária para sua execução.

3.4.1 Complexidade de tempo

Em cada chamada da função `multiplicaRecursivo`, o algoritmo divide as duas matrizes em quatro submatrizes de ordem $n/2$, realiza oito multiplicações recursivas e combina os resultados obtidos.

A criação das submatrizes exige a cópia de elementos das matrizes originais, resultando em um custo de $\Theta(n^2)$.

As oito multiplicações recursivas são realizadas sobre matrizes de ordem $n/2$, produzindo o termo $8T(n/2)$.

Por fim, as operações de soma e combinação dos blocos resultantes também possuem custo $\Theta(n^2)$. Dessa forma, a relação de recorrência do algoritmo é dada por:

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 8T(n/2) + \Theta(n^2), & \text{se } n > 1. \end{cases} \tag{9}$$

Para resolver essa recorrência, aplica-se o Teorema Mestre, considerando sua forma geral:

$$T(n) = aT(n/b) + f(n). \tag{10}$$

Identificam-se os seguintes parâmetros:

$$a = 8, \qquad b = 2, \qquad f(n) = \Theta(n^2). \tag{11}$$

Calculando o termo de comparação do Teorema Mestre:

$$n^{\log_b a} = n^{\log_2 8} = n^3. \tag{12}$$

Como $f(n) = \Theta(n^2)$, verifica-se que:

$$f(n) = O(n^{3-\varepsilon}), \tag{13}$$

para $\varepsilon = 1$.
Portanto, a recorrência satisfaz o primeiro caso do Teorema Mestre, resultando em:

$$\boxed{T(n) = \Theta(n^3)}. \tag{14}$$

Assim, o algoritmo apresenta complexidade de tempo cúbica em relação à ordem das matrizes.

3.4.2 Complexidade de espaço

A implementação utiliza alocação dinâmica para armazenar as matrizes originais, as submatrizes e os resultados intermediários das operações recursivas.

Em cada chamada, são criadas matrizes auxiliares cuja quantidade total de elementos é proporcional a n^2.

Embora sejam realizadas oito multiplicações recursivas, essas chamadas são executadas sequencialmente. Dessa forma, a memória necessária para uma chamada recursiva pode ser reutilizada após seu encerramento.

O espaço máximo utilizado pela recursão pode ser representado pela relação:

$$S(n) = S(n/2) + \Theta(n^2). \tag{15}$$

Expandindo a recorrência, obtém-se:

$$S(n) = \Theta\left(n^2 + \frac{n^2}{4} + \frac{n^2}{16} + \cdots\right). \tag{16}$$

A expressão corresponde a uma progressão geométrica de razão $1/4$. Como a soma da série geométrica é limitada por uma constante, conclui-se que:

$$\boxed{S(n) = \Theta(n^2)}. \tag{17}$$

A profundidade da recursão é $\Theta(\log n)$, sendo o espaço ocupado pela pilha de chamadas assintoticamente inferior ao espaço utilizado pelas matrizes auxiliares.

3.4.3 Análise do preenchimento com zeros

Para matrizes cuja ordem não é uma potência de dois, a implementação utiliza matrizes auxiliares de ordem:

$$m = 2^{\lceil \log_2 n \rceil}. \tag{18}$$

Para todo $n \geq 1$, verifica-se que:

$$n \leq m < 2n. \tag{19}$$

Consequentemente, $m = \Theta(n)$.

A criação e a cópia dos elementos para as matrizes auxiliares possuem custo $\Theta(m^2)$, enquanto a multiplicação recursiva apresenta custo $\Theta(m^3)$.

Como $m = \Theta(n)$, o preenchimento com zeros não altera as ordens assintóticas de tempo e espaço.

Portanto, considerando o algoritmo completo, obtêm-se as seguintes complexidades:

$$\boxed{T(n) = \Theta(n^3)} \tag{20}$$

$$\boxed{S(n) = \Theta(n^2)}. \tag{21}$$

3.5 Experimentos e resultados

Para verificar o funcionamento do algoritmo de multiplicação de matrizes, foram realizados testes com diferentes dimensões e configurações de entrada.

Os testes tiveram como objetivo avaliar a correção dos resultados produzidos pela implementação, incluindo o caso base da recursão, matrizes de ordem que não é potência de dois e matrizes de dimensões maiores.

3.5.1 Metodologia dos testes

Os testes foram organizados em arquivos de entrada e saída, armazenados no diretório `tests` do projeto.

Para cada caso, foi definido um arquivo contendo a ordem das matrizes e seus elementos, juntamente com outro arquivo contendo o resultado esperado da multiplicação.

A execução dos testes foi automatizada por meio de um `Makefile`, utilizando o comando:

```
make test
```

O procedimento executa o programa para cada arquivo de entrada e compara a saída produzida com o resultado esperado, utilizando a ferramenta `diff`.

Um teste é considerado aprovado quando os dois arquivos apresentam o mesmo conteúdo.

3.5.2 Casos de teste

Foram utilizados quatro casos de teste, apresentados na Tabela 1.

Tabela 1: Casos de teste da multiplicação de matrizes.

Teste	Ordem	Descrição	Resultado
01	2	Multiplicação de duas matrizes com elementos constantes.	Aprovado
02	1	Verificação do caso base da recursão.	Aprovado
03	3	Multiplicação com preenchimento de zeros.	Aprovado
04	4	Multiplicação de uma matriz pela matriz identidade.	Aprovado

3.5.3 Teste 01 – Matrizes de ordem 2

O primeiro teste utiliza duas matrizes de ordem 2, definidas por:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}. \tag{22}$$

O resultado esperado é:

$$C = A \cdot B = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}. \tag{23}$$

Esse teste verifica a multiplicação recursiva de matrizes cuja ordem é uma potência de dois.

3.5.4 Teste 02 – Caso base

O segundo teste utiliza matrizes de ordem 1:

$$A = \begin{bmatrix} 5 \end{bmatrix}, \qquad B = \begin{bmatrix} 3 \end{bmatrix}. \tag{24}$$

O resultado esperado é:

$$C = \begin{bmatrix} 15 \end{bmatrix}. \tag{25}$$

Esse teste verifica o caso base da recursão, no qual a multiplicação é realizada diretamente entre os únicos elementos das matrizes.

3.5.5 Teste 03 – Ordem não potência de dois

O terceiro teste utiliza matrizes de ordem 3:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad B = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}. \tag{26}$$

O resultado esperado é:

$$C = \begin{bmatrix} 30 & 24 & 18 \\ 84 & 69 & 54 \\ 138 & 114 & 90 \end{bmatrix}. \tag{27}$$

Esse caso verifica o procedimento de preenchimento com zeros, pois as matrizes são ampliadas internamente para ordem 4 antes da multiplicação recursiva.

3.5.6 Teste 04 – Matriz identidade

O quarto teste utiliza uma matriz de ordem 4 multiplicada pela matriz identidade.

A matriz de entrada é:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}. \tag{28}$$

Como a segunda matriz é a identidade I_4, o resultado esperado é:

$$A \cdot I_4 = A. \tag{29}$$

Esse teste verifica se o algoritmo preserva corretamente os elementos da matriz quando multiplicada pela identidade.

3.5.7 Análise dos resultados

Os quatro testes funcionais foram executados com sucesso, produzindo resultados iguais aos valores esperados. Os testes contemplam diferentes situações de execução, incluindo o caso base da recursão, matrizes com ordem potência de dois e matrizes que exigem preenchimento com zeros. Os resultados obtidos fornecem evidências da correção da implementação para os casos avaliados. Entretanto, esses testes não constituem uma demonstração formal de correção para todas as entradas possíveis. A análise assintótica apresentada na Seção 3.4 estabelece o comportamento teórico do algoritmo, enquanto os testes realizados verificam seu funcionamento em entradas específicas.

3.6 Considerações da tarefa

A implementação da multiplicação de matrizes por divisão e conquista permitiu aplicar os conceitos de recursividade, decomposição de problemas e análise de complexidade estudados na disciplina de Projeto e Análise de Algoritmos. A utilização de um Tipo Abstrato de Dados (TAD) para representar as matrizes contribuiu para a organização do código, separando as operações de criação, leitura, impressão e liberação de memória da lógica de multiplicação.

A estratégia adotada consiste na divisão das matrizes em quatro submatrizes, seguida da realização de oito multiplicações recursivas e da combinação dos resultados parciais. Uma decisão importante da implementação foi a utilização de preenchimento com zeros para permitir a multiplicação de matrizes cuja ordem não é uma potência de dois. Dessa forma, o algoritmo pode receber matrizes quadradas de diferentes dimensões sem modificar sua estrutura recursiva. A análise teórica demonstrou que a solução apresenta complexidade de tempo $\Theta(n^3)$ e complexidade de espaço $\Theta(n^2)$. Os testes funcionais realizados apresentaram os resultados esperados para os casos avaliados, incluindo matrizes de ordem 1, matrizes de ordem potência de dois e matrizes que exigem preenchimento com zeros.

Como limitação, destaca-se a necessidade de alocar e copiar diversas submatrizes temporárias durante a execução, o que introduz custos adicionais de memória e processamento. Embora a estratégia de divisão e conquista organize a solução de forma recursiva, a implementação com oito multiplicações por etapa mantém a mesma ordem de complexidade de tempo do algoritmo convencional de multiplicação de matrizes. Portanto, a solução desenvolvida atende ao objetivo de aplicar a técnica de divisão e conquista à multiplicação de matrizes, permitindo relacionar a implementação prática com sua análise matemática de complexidade.

4 Tarefa 2 – Título a definir

5 Tarefa 3 – Título a definir

6 Tarefa 4 – Título a definir

7 Tarefa 5 – Título a definir

8 Tarefa 6 – Título a definir

9 Tarefa 7 – Título a definir

10 Tarefa 8 – Título a definir

11 Tarefa 9 – Título a definir

12 Considerações Finais

Referências